Assignment 3

System Call

20215174 임혁준

20225087 성준호

Submit : 2025.05.26

1. 문제 분석

기존의 PintOS는 유저 프로그램의 실행과 System Call Handling에 대하여 Skeleton code만 제공하고 미완성된 상태이다. 이번 과제는 이 기능들이 User영역에서 Kernel 영역에 대한 침범을 막으며, 메모리의 누수가 없는 채로 원하는 기능을 수행할 수 있도록 구현해야 한다.

2. 알고리즘 설계 및 구현

Problem 1.

프로세스가 Exit system call을 호출하여 종료될 시 Process Termination Messages를 출력하는 기능을 구현한다. 해당 기능을 구현하기 위해서 System Call Handling 기능이 우선적으로 구현되어야 했다.

/userprog/syscall.c

```c
void
syscall_handler (struct intr_frame *f)
{
  void* sp = f->esp;
  uint32_t syscall_num = *(uint32_t*)(sp);

  switch (syscall_num)
  {
    case SYS_HALT:
      halt();
      break;

    case SYS_EXIT:
      check_user_vaddr(sp + 4);
      exit(*(int*)(sp + 4));
      break;
```

```c
void
exit(int status)
{
  struct thread* cur = thread_current();
  cur->exit_status = status;
  int i;
  for (i = 3; i < 128; i++) {
    if (cur->fd[i] != NULL)
      close(i);
  }

  printf("%s: exit(%d)\n", cur->name, status); // Process Termination Message
  thread_exit();
}
```

Problem 2.

"/bin/ls -l bar foo" 와 같은 command line을 argument로 구분하여 user program이 argument와 같이 동작할 수 있도록 하는 기능을 추가한다. 이를 위해선 command line에 대한 parsing이 필요하며 이를 유저 프로세스의 스택에 올리는 과정이 필요하다. Stanford Pintos Manual에 따르면 예시와 같은 command line은 유저 프로세스의 스택에 다음과 같이 parsing되어 올려진다는 것을 알 수 있다.

Address	Name	Data	Type
0xbffffffc	argv[3][...]	'bar\0'	char[4]
0xbffffff8	argv[2][...]	'foo\0'	char[4]
0xbffffff5	argv[1][...]	'-l\0'	char[3]
0xbffffffed	argv[0][...]	'/bin/ls\0'	char[8]
0xbffffffec	word-align	0	uint8_t
0xbffffffe8	argv[4]	0	char *
0xbffffffe4	argv[3]	0xbffffffc	char *
0xbffffffe0	argv[2]	0xbffffff8	char *
0xbffffffdc	argv[1]	0xbffffff5	char *
0xbffffffd8	argv[0]	0xbffffffed	char *
0xbffffffd4	argv	0xbffffffd8	char **
0xbffffffd0	argc	4	int
0xbffffffcc	return address	0	void (*) ()

Fig. Pintos User Stack

따라서 Argument Passing은 command line을 parsing하여 user process stack space에 Fig와 같은 형태로 push하는 것이 주된 목적이다.

이를 위해선 PintOS가 user program을 실행시키는 과정에 대한 파악이 필요하다. 먼저 PintOS는 process_excute(const char *file_name) 함수를 통해 유저 프로세스를 실행시키는 과정을 시작한다. 이 함수에서 user program의 name을 가지는 thread가 생성되며, 생성된 thread는 start_process( ) 함수를 실행하는데, 해당 함수에서 프로세스의 공간이 메모리에 할당된다. 그 과정에서 load( ) 함수가 유저의 stack 공간의 시작 주소를 반환하는 것을 파악할 수 있었고, 해당 주소를 활용하여 stack을 push하는 것이 가능함을 파악할 수 있었다. 따라서 Argument Passing은 start_process( ) 함수에 command line을 parsing하고 이를 stack에 쌓는 기능을 추가하여 구현하고자 하였다.

/userprog/process.c

```c
tid_t
process_execute (const char *file_name)
{
  char *fn_copy;
  tid_t tid;

  /* Make a copy of FILE_NAME.
     Otherwise there's a race between the caller and load(). */
  fn_copy = palloc_get_page (0);
  if (fn_copy == NULL)
    return TID_ERROR;
  strlcpy (fn_copy, file_name, PGSIZE);

  ////////////////////////////////////////////////////////// ADDED

  char* tempptr;
  char* token = strtok_r(file_name, " ", &tempptr);

  if (filesys_open(token) == NULL)
      return -1;

  ////////////////////////////////////////////////////////// ADDED DONE
```

Process_excute( ) 함수에서는 받아온 command line의 첫번째 인자를 program의 name으로 활용할 수 있게하는 parsing을 진행하였다. 이를 구현하기 위해 PintOS의 library에 구현되어 있는 string.c의 strtok_r( ) 함수를 사용하였다. 해당 함수는 주어진 문자열에 주어진 delimeter가 첫번째로 위치하는 곳을 '₩0'로 치환하고 해당 문자열의 주소를 다시 반환하는 함수이다. 따라서 file_name(command line)의 첫번째 공백까지를 parsing하여 token에 저장하였고, 이를 통해 thread를 생성하도록 하였다.

```c
static void
start_process (void *file_name_)
{
  char *file_name = file_name_;
  struct intr_frame if_;
  bool success;

  /* Initialize interrupt frame and load executable. */
  memset (&if_, 0, sizeof if_);
  if_.gs = if_.fs = if_.es = if_.ds = if_.ss = SEL_UDSEG;
  if_.cs = SEL_UCSEG;
  if_.eflags = FLAG_IF | FLAG_MBS;

  ////////////////////////////////////////////////////////// ADDED
  /* Count tokens. */
  char* ptr;
  char* rest;
  char* token;
  int argc = 0;
  int i;

  char* file_name_to_count = malloc(strlen(file_name) + 1);
  strlcpy(file_name_to_count, file_name, strlen(file_name) + 1);

  ptr = file_name_to_count;

  do
  {
      token = strtok_r(ptr, " ", &rest);
      argc++;
      ptr = rest;
  }
  while (token != NULL);

  argc--;
  free(file_name_to_count);

  /* Save argv. */
  char** argv;
  argv = (char**)malloc(sizeof(char*) * argc);
  ptr = file_name;

  for (i = 0; i < argc; i++)
  {
      token = strtok_r(ptr, " ", &rest);
      argv[i] = token;
      ptr = rest;
  }
```

```c
success = load (argv[0], &if_.eip, &if_.esp);
sema_up(&thread_current()->parent->load_lock);

/* Push to stack. */
if (success)
{
    void** esp = &if_.esp;
    int length = 0;

    /* Push command to stack. */
    for (i = argc - 1; i >= 0; i--)
    {
        length = strlen(argv[i]) + 1;
        *esp -= length;
        memcpy(*esp, argv[i], length);
        argv[i] = *esp;
    }

    /* Push word-align finished. */
    while ((PHYS_BASE - *esp) % 4 != 0) {
        *esp -= sizeof(uint8_t);
        **(uint8_t**)esp = 0;
    }

    /* Push NULL. */
    *esp -= 4;
    *(uint8_t*)(*esp) = 0;

    /* Push addresses of argv. */
    for (i = argc - 1; i >= 0; i--)
    {
        *esp -= sizeof(uint32_t**);
        *(uint32_t**)*esp = argv[i];
    }
```

```c
    /* Push start address of argv. */
    *esp -= sizeof(uint32_t**);
    *(uint32_t*)*esp = *esp + 4;

    /* Push value of argc. */
    *esp -= sizeof(uint32_t);
    *(uint32_t*)*esp = argc;

    /* Push return address. */
    *esp -= 4;
    *(uint32_t*)*esp = 0;
}
////////////////////////////////////////////////////////////////////////// ADDED DONE
```

start_process( ) 함수에서는 동일한 방식으로 strtok_r( ) 함수를 활용하여 thread_create( )의 aux에 저장되어 있는 command line의 복사본을 parsing하여 stack에 Fig와 같은 형태로 push하는 기능을 추가하였다. esp에는 load( ) 함수를 통해 받는 stack space의 시작 주소를 저장하고, 이를 활용하여 push를 진행한다.

Problem 3.

미구현된 System Call을 구현한다. 추가적으로, 포인터 주소 영역이 user영역인지, kernel 영역인 확인하는 코드를 추가해 잘못된 포인터를 추적하면 exit(-1)을 반환한다. 추가적으로, NULL값이 들어오면 안되는 인자에는 예외를 추가해준다.

/userprog/syscall.c

-void halt(void)

```
case SYS_HALT:
    halt();
    break;
```

```
void
halt(void)
{
    shutdown_power_off();
}
```

PintOS에서 지원하는 shutdown_power_off() 함수를 사용해 간단히 구현한다.

-void exit(int)

```
case SYS_EXIT:
    check_user_vaddr(sp + 4);
    exit(*(int*)(sp + 4));
    break;
```

```
void
exit(int status)
{
    struct thread* cur = thread_current();
    cur->exit_status = status;
    int i;
    for (i = 3; i < 128; i++) {
        if (cur->fd[i] != NULL)
            close(i);
    }

    printf("%s: exit(%d)\n", cur->name, status); // Process Termination Message
    thread_exit();
}
```

부모가 자식의 exit_status를 참조하기 위해, 기존 SYS_EXIT 구현에서 thread에 exit status를 추가하는 코드를 삽입한다. 이를 위해 thread.h의 thread 구조체에 int exit_status가 선언된다. 메모리 사용을 줄이기 위해 thread가 exit될 때, file_descriptor를 모두 close() 해주어야 한다.

-pid_t exec(char*)

```
case SYS_EXEC:
    check_user_vaddr(sp + 4);
    f->eax = exec((const char*)*(uint32_t*)(sp + 4));
    break;
```

```c
pid_t
exec(const char* file)
{
    char *fn_copy = palloc_get_page(0);
    if (fn_copy == NULL)
        exit(-1);
    strlcpy(fn_copy, file, PGSIZE);
    pid_t tid = process_execute(fn_copy);
    palloc_free_page(fn_copy);
    return tid;
}
```

process_execute()함수를 사용해 exec를 실행한다. 유저가 넘긴 원래 문자열을 전달했다 원래 thread가 종료되는 것을 방지하기 위해, 문자열 file을 fn_copy로 안전히 복사하고, overflow 방지를 위해 strlcpy()를 이용한다.

-int wait(pid_t)

```c
case SYS_WAIT:
    check_user_vaddr(sp + 4);
    f->eax = wait((pid_t)*(uint32_t*)(sp + 4));
    break;
```

```c
int
wait(pid_t pid)
{
    return process_wait(pid);
}
```

process_wait()함수를 사용해 wait를 실행한다.

-bool create(char*,unsigned)

```c
case SYS_CREATE:
    check_user_vaddr(sp + 4);
    f->eax = create((const char*)*(uint32_t*)(sp + 4), (unsigned)*(uint32_t*)(sp + 8));
    break;
```

```
bool
create(const char* file, unsigned initial_size)
{
    if (file == NULL)
        exit(-1);
    return filesys_create(file, initial_size);
}
```

file이 NULL이라면 exit(-1)을 실행한다. 아니라면, filesys_create()함수를 이용해 create를 실행한다.

-bool remove(char*)

```
case SYS_REMOVE:
    check_user_vaddr(sp + 4);
    f->eax = remove((const char*)*(uint32_t*)(sp + 4));
    break;
```

```
bool
remove(const char* file)
{
    if (file == NULL)
        exit(-1);
    return filesys_remove(file);
}
```

file이 NULL이라면 exit(-1)을 실행한다. 아니라면, filesys_remove()함수를 사용해 remove를 실행한다.

-int open(char*)

```
case SYS_OPEN:
    check_user_vaddr(sp + 4);
    f->eax = open((const char*)*(uint32_t*)(sp + 4));
    break;
```

```c
int
open(const char* file)
{
    if (file == NULL)
        exit(-1);

    lock_acquire(&file_lock);
    struct file* return_file = filesys_open(file);
    if (return_file == NULL) {
        lock_release(&file_lock);
        return -1;
    }

    int i;
    for (i = 3; i < 128; i++)
    {
        if (getfile(i) == NULL)
        {
            if (strcmp(thread_current()->name, file) == false)
                file_deny_write(return_file);
            thread_current()->fd[i] = return_file;
            lock_release(&file_lock);
            return i;
        }
    }
    lock_release(&file_lock);
    return -1;
}
```

file이 NULL이라면 exit(-1)을 실행한다. 위 과정을 file_lock을 선점한 상태에서 실행해야 한다. return할 때, lock을 release한다. filesys_open() 함수를 호출해 file을 연 뒤, NULL값이라면 즉시 반환한다. 아니라면, 현재 thread의 file_descriptor에 전달한다.

-int filesize(int)

```c
case SYS_FILESIZE:
    check_user_vaddr(sp + 4);
    f->eax = filesize((int)*(uint32_t*)(sp + 4));
    break;
```

```c
int
filesize(int fd)
{
    struct file* f = getfile(fd);
    if (f == NULL)
        exit(-1);
    else
        return file_length(f);
}
```

file_descriptor로부터 파일을 얻을 수 없다면 즉시 exit(-1)한다. 아니라면, file_length()를 반환한다.

-int read(int,void*,unsigned)

```c
case SYS_READ:
    check_user_vaddr(sp + 4);
    f->eax = read((int)*(uint32_t*)(sp + 4), (void*)*(uint32_t*)(sp + 8), (unsigned)*((uint32_t*)(sp + 12)));
    break;
```

```c
int
read(int fd, void* buffer, unsigned size)
{
    check_user_vaddr(buffer);
    lock_acquire(&file_lock);
    if (fd == 0)
        return 0;
    else
    {
        struct file* f = getfile(fd);
        if (f == NULL)
        {
            lock_release(&file_lock);
            exit(-1);
        }
        lock_release(&file_lock);
        return file_read(f, buffer, size);
    }
}
```

file_descriptor의 값이 0인 경우 STD_IN(user input)으로부터 데이터를 받아오는 것이기에, 따로 buffer에 PintOS에서 정의된 input_getc( ) 함수를 통해 데이터를 저장해야 하지만, test case에선 이를 필요로 하지 않았기에 구현하지 않았다. 나머지 값의 경우 file_read( ) 함수를 통해 file의 내용을 read한다. 이 과정은 file_lock을 선점한 상태에서 실행해야 하며, return할 때, lock을 release한다.

-int write(int,void*,unsigned)

```c
case SYS_WRITE:
    check_user_vaddr(sp + 4);
    f->eax = write((int)*(uint32_t*)(sp + 4), (void*)*(uint32_t*)(sp + 8), (unsigned)*((uint32_t*)(sp + 12)));
    break;
```

```c
int
write(int fd, const void* buffer, unsigned size)
{
    check_user_vaddr(buffer);
    lock_acquire(&file_lock);
    if (fd == 1)
    {
        putbuf(buffer, size);
        lock_release(&file_lock);
        return size;
    }
    else
    {
        struct file* f = getfile(fd);
        if (f == NULL)
        {
            lock_release(&file_lock);
            exit(-1);
        }
        int ret = file_write(f, buffer, size);
        lock_release(&file_lock);
        return ret;
    }
}
```

file_descriptor의 값이 1인 경우 STD_OUT에 데이터를 작성하기에, PintOS의 putbuf( ) 함수를 통해 STD_OUT에 입력을 진행한다. 나머지 값의 경우 file_write( ) 함수를 통해 file에 데이터를 write한다. 이 과정은 file_lock을 선점한 상태에서 실행해야 하며, return할 때, lock을 release한다.

-void seek(int,unsigned)

```
case SYS_SEEK:
    check_user_vaddr(sp + 4);
    seek((int)*(uint32_t*)(sp + 4), (unsigned)*((uint32_t*)(sp + 8)));
    break;
```

```
void
seek(int fd, unsigned position)
{
    struct file* f = getfile(fd);
    if (f == NULL)
        exit(-1);
    else
        return file_seek(f, position);
}
```

syscall.c

file_descriptor로부터 파일을 얻을 수 없다면 즉시 exit(-1)한다. 아니라면, file_seek()를 반환한다.

-unsigned tell(int)

```
case SYS_TELL:
    check_user_vaddr(sp + 4);
    f->eax = tell((int)*(uint32_t*)(sp + 4));
    break;
```

```
unsigned
tell(int fd)
{
    struct file* f = getfile(fd);
    if (f == NULL)
        exit(-1);
    else
        return file_tell(f);
}
```

syscall.c

file_descriptor로부터 파일을 얻을 수 없다면 즉시 exit(-1)한다. 아니라면, file_tell()를 반환한다.

-void close(int)

```
case SYS_CLOSE:
    check_user_vaddr(sp + 4);
    close((int)*(uint32_t*)(sp + 4));
    break;
```

```
void
close(int fd)
{
    lock_acquire(&file_lock);
    struct file* f = getfile(fd);
    if (f == NULL) {
        lock_release(&file_lock);
        exit(-1);
    }
    file_close(f);
    thread_current()->fd[fd] = NULL;
    lock_release(&file_lock);
}
```

syscall.c

file_descriptor로부터 파일을 얻을 수 없다면 즉시 exit(-1)한다. 아니라면, file_close() 후 현재 thread의 file_descriptor를 초기화한다.

추가 구현 함수

-struct file* getfile(int)

```
struct file*
getfile(int fd)
{
    return (thread_current()->fd[fd]);
}
```

현재 thread의 file_descriptor로부터 파일을 얻는다. struct file은 filesys/file.h가 아닌 .c에 선언되어있어 참조할 수 없기에 syscall.c에 struct file을 복사하여야 한다.

-void check_user_vaddr(const)

```
void
check_user_vaddr(const void* vaddr)
{
    if (!is_user_vaddr(vaddr))
        exit(-1);
}
```

참조하고자 하는 주소가 user영역이 아니라면 exit(-1)를 반환한다.

메모리 및 동기화 관련

/userprog/process.c

-tid_t process_execute(char*)

```c
tid_t
process_execute (const char *file_name)
{
  char *fn_copy;
  tid_t tid;

  /* Make a copy of FILE_NAME.
     Otherwise there's a race between the caller and load(). */
  fn_copy = palloc_get_page (0);
  if (fn_copy == NULL)
    return TID_ERROR;
  strlcpy (fn_copy, file_name, PGSIZE);

  ////////////////////////////////////////////////////////////////////// ADDED

  char* tempptr;
  char* token = strtok_r(file_name, " ", &tempptr);

  if (filesys_open(token) == NULL)
      return -1;

  ////////////////////////////////////////////////////////////////////// ADDED DONE
  /* Create a new thread to execute command. */
  struct thread* cur = thread_current();
  tid = thread_create (token, PRI_DEFAULT, start_process, fn_copy);
  sema_down(&cur->load_lock);
  if (tid == TID_ERROR)
    palloc_free_page (fn_copy);

  ////////////////////////////////////////////////////////////////////// ADDED

  struct list_elem* e;
  struct thread* t = NULL;
  for (e = list_begin(&cur->child); e != list_end(&cur->child); e = list_next(e)) {
      t = list_entry(e, struct thread, child_elem);
      if (t->exit_status == -1) {
          return process_wait(tid);
      }
  }

  ////////////////////////////////////////////////////////////////////// ADDED DONE

  return tid;
}
```

process.c

file을 열 수 없다면, return -1한다. 자식 프로세스가 성공적으로 load하기 전까지 부모는 대기해야 한다. 이를 위해 sema_down(load_lock)을 호출해 자식이 load되기 전까지 기다린다. 만약 비정상적으로 종료된 자식프로세스가 있다면 자식의 종료상태를 회수하기 위해 process_wait(tid)를 호출한다.

-int process_wait(tid_t)

```c
int
process_wait (tid_t child_tid)
{
    ////////////////////////////////////////////////////////////////////// ADDED

    struct list_elem* e;
    struct thread* t = NULL;
    int exit_status;

    for (e = list_begin(&(thread_current()->child)); e != list_end(&(thread_current()->child)); e = list_next(e)) {
        t = list_entry(e, struct thread, child_elem);
        if (child_tid == t->tid) {
            sema_down(&(t->child_lock));
            exit_status = t->exit_status;
            list_remove(&(t->child_elem));
            sema_up(&(t->mem_lock));
            return exit_status;
        }
    }
    return -1;

    ////////////////////////////////////////////////////////////////////// ADDED DONE
}
```

tid와 일치하는 자식이 있다면, 해당 자식의 exit를 기다리고, exit status를 반환한다. 이후 자식을 리스트에게 지우고, 자식이 memory에서 정리될 수 있도록 lock을 한다. 만약 tid와 일치하는 자식이 없다면, -1을 반환한다.

-int process_exit(void)

```c
/* Free the current process's resources. */
void
process_exit (void)
{
  struct thread *cur = thread_current ();
  uint32_t *pd;

  /* Destroy the current process's page directory and switch back
     to the kernel-only page directory. */
  pd = cur->pagedir;
  if (pd != NULL)
    {
      /* Correct ordering here is crucial.  We must set
         cur->pagedir to NULL before switching page directories,
         so that a timer interrupt can't switch back to the
         process page directory.  We must activate the base page
         directory before destroying the process's page
         directory, or our active page directory will be one
         that's been freed (and cleared). */
      cur->pagedir = NULL;
      pagedir_activate (NULL);
      pagedir_destroy (pd);
    }

  ////////////////////////////////////////////////////////////////////// ADDED

  sema_up(&(cur->child_lock));
  sema_down(&(cur->mem_lock));

  ////////////////////////////////////////////////////////////////////// ADDED DONE
}
```

자식 프로세스가 종료되기 전 lock을 해제해 부모의 대기가 끝나도록 해준다. 부모가 자신을 리스트에서 완전히 제거하기 전 먼저 process가 종료되지 않도록 memory에서 해제되기 전까지 lock을 기다린다.

/userprog/exception.c

-static void page_fault(struct intr_frame)

```
static void
page_fault (struct intr_frame *f)
{
  .
  .
  .

  ////////////////////////////////////////////////////////////////////// ADDED

  if (!user || is_kernel_vaddr(fault_addr) || not_present)
  {
      exit(-1);
  }

  ////////////////////////////////////////////////////////////////////// ADDED DONE

  /* To implement virtual memory, delete the rest of the function
     body, and replace it with code that brings in the page to
     which fault_addr refers. */
  printf ("Page fault at %p: %s error %s page in %s context.\n",
          fault_addr,
          not_present ? "not present" : "rights violation",
          write ? "writing" : "reading",
          user ? "user" : "kernel");
  kill (f);
}
```

exception.c

Page fault 중 일부 예외는 exit(-1)로 처리되도록 추가한다.

/threads/thead.c, thread.h

-struct thread

```
#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t *pagedir;                  /* Page directory. */
    struct semaphore child_lock;
    struct semaphore mem_lock;
    struct semaphore load_lock;
    struct thread* parent;
    struct list child;
    struct list_elem child_elem;
    int exit_status;
    struct file* fd[128];
#endif
```

thread.h

위에서 필요로 한 lock과, 자식 프로세스을 저장할 배열을 선언한다. exit status를 저장할 변수와 descriptor, 부모를 저장할 struct thread*를 선언한다.

```
static void
init_thread (struct thread *t, const char *name, int priority)
{
  ASSERT (t != NULL);
  ASSERT (PRI_MIN <= priority && priority <= PRI_MAX);
  ASSERT (name != NULL);

  memset (t, 0, sizeof *t);
  t->status = THREAD_BLOCKED;
  strlcpy (t->name, name, sizeof t->name);
  t->stack = (uint8_t *) t + PGSIZE;
  t->priority = priority;
  t->magic = THREAD_MAGIC;
  list_push_back (&all_list, &t->allelem);

#ifdef USERPROG
    int i;
    for (i = 0; i < 128; i++)
        t->fd[i] = NULL;
    sema_init(&(t->child_lock), 0);
    sema_init(&(t->mem_lock), 0);
    sema_init(&(t->load_lock), 0);
    t->parent = running_thread();
    list_init(&(t->child));
    list_push_back(&(running_thread()->child), &(t->child_elem));
#endif
}
```

thread.c

semaphore를 init_thread()에서 0으로 초기화해준다. 또한, thread가 생성될 때, 현재 실행중인 thread를 부모로 두도록 설정한다.

3. 결과

-실행결과

```
pass tests/userprog/args-none
pass tests/userprog/args-single
pass tests/userprog/args-multiple
pass tests/userprog/args-many
pass tests/userprog/args-dbl-space
pass tests/userprog/sc-bad-sp
pass tests/userprog/sc-bad-arg
pass tests/userprog/sc-boundary
pass tests/userprog/sc-boundary-2
pass tests/userprog/halt
pass tests/userprog/exit
pass tests/userprog/create-normal
pass tests/userprog/create-empty
pass tests/userprog/create-null
pass tests/userprog/create-bad-ptr
pass tests/userprog/create-long
pass tests/userprog/create-exists
pass tests/userprog/create-bound
pass tests/userprog/open-normal
pass tests/userprog/open-missing
pass tests/userprog/open-boundary
pass tests/userprog/open-empty
pass tests/userprog/open-null
pass tests/userprog/open-bad-ptr
pass tests/userprog/open-twice
pass tests/userprog/close-normal
pass tests/userprog/close-twice
pass tests/userprog/close-stdin
pass tests/userprog/close-stdout
pass tests/userprog/close-bad-fd
pass tests/userprog/read-normal
pass tests/userprog/read-bad-ptr
pass tests/userprog/read-boundary
pass tests/userprog/read-zero
pass tests/userprog/read-stdout
pass tests/userprog/read-bad-fd
pass tests/userprog/write-normal
pass tests/userprog/write-bad-ptr
pass tests/userprog/write-boundary
pass tests/userprog/write-zero
pass tests/userprog/write-stdin
pass tests/userprog/write-bad-fd
pass tests/userprog/exec-once
pass tests/userprog/exec-arg
```

```
pass tests/userprog/exec-multiple
pass tests/userprog/exec-missing
pass tests/userprog/exec-bad-ptr
pass tests/userprog/wait-simple
pass tests/userprog/wait-twice
pass tests/userprog/wait-killed
pass tests/userprog/wait-bad-pid
pass tests/userprog/multi-recurse
pass tests/userprog/multi-child-fd
pass tests/userprog/rox-simple
pass tests/userprog/rox-child
pass tests/userprog/rox-multichild
pass tests/userprog/bad-read
pass tests/userprog/bad-write
pass tests/userprog/bad-read2
pass tests/userprog/bad-write2
pass tests/userprog/bad-jump
pass tests/userprog/bad-jump2
pass tests/userprog/no-vm/multi-oom
pass tests/filesys/base/lg-create
pass tests/filesys/base/lg-full
pass tests/filesys/base/lg-random
pass tests/filesys/base/lg-seq-block
pass tests/filesys/base/lg-seq-random
pass tests/filesys/base/sm-create
pass tests/filesys/base/sm-full
pass tests/filesys/base/sm-random
pass tests/filesys/base/sm-seq-block
pass tests/filesys/base/sm-seq-random
pass tests/filesys/base/syn-read
pass tests/filesys/base/syn-remove
pass tests/filesys/base/syn-write
All 76 tests passed.
```

실행 결과에 따르면, 76개의 테스트를 모두 성공적으로 pass하였다.